

Buổi 05 (3 tiết) – Chương 2: Quy trình thiết kế & Build + Bài tập/Lab (2.3–2.7)

1.1 2.3 Quy trình thiết kế ứng dụng React Native

Khi một ứng dụng React Native được khởi chạy, nó sẽ có 3 luồng chính như sau:

Native thread (Native Queue): Đây là thread chính được khởi tạo ngay sau khi ứng dụng bắt đầu chạy. Nó có nhiệm vụ kích hoạt Javascript thread để thực thi code React mà các lập trình viên đã tạo ra trước đó và sau đó gửi lại cho Native thread để thực thi chúng. Đồng thời thread này cũng có nhiệm vụ lắng nghe các sự kiện thao tác trên màn hình và gửi sang cho Javascript thread. Do 2 thread này luôn giao tiếp liên mạch với nhau nên khi giao diện ứng dụng cần cập nhật thì việc thực hiện khá là nhanh chóng.

Javascript thread (JS Queue): Thread này có nhiệm vụ thực thi và đóng gói code javascript chưa logic nghiệp vụ của app và sau đó gửi đến cho Native thread.

Shadow thread: Đây là nơi tính toán lại bố cục giao diện được hiển thị trên màn hình ứng dụng của bạn. Facebook sử dụng một công cụ là Yoga để thực hiện công việc tính toán này và gửi lại cho Native thread để hiển thị lên màn hình.

Trước khi release app android hoặc ios với react-native thường chúng ta sẽ run 2 câu lệnh `npx react-native run-android --variant=release` hoặc `npx react-native run-ios --configuration`

Release. Tiến trình này sẽ đóng gói toàn bộ code javascript của bạn thành 1 file là `main.bundle.js` bằng NPM của [NodeJS](#). Khi một app start, Native thread sẽ khởi động và sau đó gửi một message qua một bridge là JSI để kích hoạt Javascript thread. Sau đó Javascript thread sẽ gửi lại toàn logic UI/UX cần hiển thị cho Native thread thông qua JSI. JSI sẽ gọi đến Shadow thread và sử dụng [Yoga](#) để tính toán bố cục layout. Cuối cùng JSC sẽ gửi lại cho Native thread để hiển thị lên màn hình ứng dụng.

Nếu bạn muốn tạo View và Text để hiển thị lên màn hình thì với React Native mình có ví dụ sau:

Ở phía react-native chúng ta tạo ra một giao diện hiển thị như sau:

```
import React, { Component } from "react";

import { Text, View, AppRegistry } from "react-native";

function HelloWorldApp() {

  render() {

    return (

      <View style={{ padding: 40 }}>

        <Text>Hello world!</Text>

      </View>

    );

  }

}

export default HelloWorldApp;
```

```
AppRegistry.registerComponent("HelloWorldApp", () => HelloWorldApp);
```

Sau đó JS thread sẽ gửi cho Native thread một đoạn mã như sau:

```
[ [2,3,[2,'Text',{...}]] [2,3,[3,'View',{...}]] ]
```

Lúc này với Native thread sẽ khởi tạo các object view với Android View -> ViewManager, TextView -> TextView và iOS View -> RCTViewManager, TextView -> UITextView

Ví dụ:

- **Android:** Text -> new TextView(getContext())
- **IOS:** Text -> UITextView(frame: CGRect(x: 20.0, y: 90.0, width: 250.0, height: 100.0))

Nếu các bạn đã làm việc với Android chắc sẽ không còn xa lạ với việc tạo View như thế này đúng không?

```
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"

    xmlns:tools="http://schemas.android.com/tools"

    android:layout_width="match_parent"

    android:layout_height="match_parent"

    android:paddingBottom="@dimen/activity_vertical_margin"

    android:paddingLeft="@dimen/activity_horizontal_margin"

    android:paddingRight="@dimen/activity_horizontal_margin"

    android:paddingTop="@dimen/activity_vertical_margin"

    tools:context=".MainActivity">
```

```
<TextView

    android:id="@+id/simpleTextView"

    android:layout_width="wrap_content"

    android:layout_height="wrap_content"

    android:layout_centerHorizontal="true"

    android:text="Before Clicking"

    android:textColor="#f00"

    android:textSize="25sp"

    android:textStyle="bold|italic"

    android:layout_marginTop="50dp"/>

</RelativeLayout>
```

Nó là một file XML cho phép chúng ta tạo ra UI trên Android. Bây giờ dựa vào message của Javascript thread gửi sang, Native thread sẽ khởi tạo các view này cho chúng ta bằng Java/Kotlin.

1.2 2.4 Hướng dẫn build ứng dụng trên máy thiết bị di động

A. Chuẩn bị thiết bị Android

1. Bật chế độ nhà phát triển (Developer mode)

- Vào Cài đặt → Giới thiệu điện thoại → Nhấn nhiều lần vào “Số phiên bản” để kích hoạt Developer Mode.

2. **Bật USB Debugging**

- Vào Cài đặt → Tùy chọn nhà phát triển → Bật **Gỡ lỗi USB (USB Debugging)**

3. **Kết nối thiết bị với máy tính qua cáp USB**

- 4. **Cho phép máy tính tin cậy (Allow USB debugging)** khi được hỏi trên điện thoại.

B. Kiểm tra kết nối thiết bị

Mở terminal/cmd và gõ:

```
bash
```

```
CopyEdit
```

```
adb devices
```

Nếu thấy danh sách thiết bị hiện ra như:

```
arduino
```

```
CopyEdit
```

```
List of devices attached
```

```
8AADB12345          device
```

nghĩa là thiết bị đã kết nối thành công.

C. Build và chạy ứng dụng React Native

Trong thư mục dự án:

```
bash
```

```
CopyEdit
```

```
npx react-native run-android
```

Thiết bị phải được mở khóa trong suốt quá trình build.

II. Build và chạy trên thiết bị iOS (chỉ khả dụng trên macOS)

A. Chuẩn bị thiết bị iPhone

1. Cài đặt Xcode (từ App Store)
2. Cắm iPhone vào máy Mac
3. **Bật Developer Mode trên iPhone** (iOS 16+ sẽ hỏi khi lần đầu kết nối với Xcode)
4. **Tin cậy thiết bị và cấp quyền trên iPhone**

B. Cấu hình Xcode

1. Mở thư mục iOS trong dự án:

```
bash
```

```
CopyEdit
```

```
open ios/MyApp.xcworkspace
```

2. Chọn Team (Apple ID) trong phần Signing & Capabilities
3. Đảm bảo **Device target** là iPhone thật

C. Build và chạy ứng dụng

Quay lại terminal:

```
bash
```

```
CopyEdit
```

```
npx react-native run-ios --device
```

Hoặc chạy trực tiếp từ Xcode: chọn thiết bị → nhấn nút  để chạy.

2.5 Câu hỏi ôn tập

8. Bạn sẽ tổ chức cấu trúc thư mục dự án RN như thế nào để dễ bảo trì?
7. Nêu các lưu ý khi build trên thiết bị thật (USB debugging, permissions...).
6. Giải thích ý nghĩa của build debug và build release.
5. Nêu quy trình phát triển 1 app: phân tích yêu cầu → thiết kế → triển khai → kiểm thử → phát hành.
4. Trình bày cách cấu hình chạy trên iOS và giới hạn khi dùng Windows.
3. Trình bày các bước cài môi trường chạy Android (SDK, emulator, adb).
2. Expo và React Native CLI khác nhau ở điểm nào? Khi nào chọn mỗi cách?
1. Mô tả luồng render UI trong React Native ở mức khái niệm.

2.6 Bài tập thực hành

4. Viết checklist quy trình build release (Android) cho dự án của bạn.
3. Thiết kế wireframe đơn giản cho app ghi chú: danh sách + thêm + chi tiết.
2. Tạo skeleton app 3 màn hình (Home, Profile, Settings) dùng điều hướng (navigation) cơ bản.
1. Cài đặt môi trường Android (SDK + emulator) và chạy một app mẫu.

2.7 Phiếu bài tập thực hành (Mẫu 04)

Sinh viên thực hiện theo yêu cầu, chụp màn hình/ghi video demo và nộp mã nguồn theo hướng dẫn của giảng viên.

Tên bài thực hành	Chương 2 – Lab 02: Thiết lập môi trường & tạo cấu trúc đa màn hình
--------------------------	--

Mục tiêu	Thiết lập môi trường, chạy được app trên emulator/thiết bị và tạo cấu trúc app đa màn hình.
-----------------	---

Thời lượng	120–180 phút
gợi ý	
Chuẩn bị	Android Studio + SDK (hoặc cài SDK tools). Thiết bị Android bật Developer options + USB debugging (nếu chạy máy thật).
Nhiệm vụ	Tạo project RNMultiScreen.
thực hành	Cài React Navigation và tạo 3 màn hình: Home, Profile, Settings. Thêm header title khác nhau cho từng màn hình. Thêm một nút từ Home sang Profile và quay về. Ghi lại 3 lỗi môi trường bạn gặp (nếu có) và cách bạn đã xử lý.
Sản phẩm	Link repo.
nộp	Video ngắn (30–60s) hoặc ảnh chụp điều hướng giữa 3 màn hình. Tệp ghi chú troubleshooting (txt/md).
Tiêu chí	Chạy được trên emulator/thiết bị (30%).
đánh giá	Điều hướng hoạt động đúng (30%). Tổ chức code rõ ràng (20%). Ghi chú troubleshooting đầy đủ (20%).